

GMM, DBSCAN y OPTICS

```
library(mclust)
```

Package 'mclust' version 6.1.2

Type 'citation("mclust")' for citing this R package in publications.

```
library(dbSCAN)
```

Adjuntando el paquete: 'dbSCAN'

The following object is masked from 'package:stats':

```
as.dendrogram
```

```
library(MASS)
```

```
set.seed(42)
```

Resumen

Hasta ahora veníamos usando métodos que forman grupos de distintas maneras.

K-means piensa los grupos como puntos alrededor de un centroide. Funciona bien cuando los grupos son compactos, más o menos redondeados y de tamaño parecido.

Las mixturas gaussianas, o GMM, dan un paso más. En vez de pensar cada grupo como puntos alrededor de un centro, piensan cada grupo como una nube gaussiana. Eso permite grupos elípticos, con distinta dispersión y orientación. Además, GMM no solo asigna una observación a un grupo: también dice con qué probabilidad pertenece a cada componente.

DBSCAN cambia la lógica. No busca centros ni gaussianas. Busca zonas densas conectadas. En esta mirada, un grupo es como un cuerpo formado por puntos cercanos. Los puntos que quedan aislados no se fuerzan dentro de un grupo: quedan como ruido.

OPTICS continúa esa idea de densidad, pero evita depender tanto de un único valor fijo de `eps`. En vez de devolver directamente una partición, construye un gráfico de alcanzabilidad que muestra la estructura de densidades. Después

podemos cortar ese gráfico para extraer grupos.

La idea general queda así:

k-means -> grupos alrededor de centroides
GMM -> grupos como nubes gaussianas
DBSCAN -> grupos como zonas densas conectadas
OPTICS -> mapa de estructura de densidades

La pregunta no es cuál método es mejor en abstracto. La pregunta es qué idea de grupo se ajusta mejor a la forma de los datos.

1. GMM — Mixturas Gaussianas

1.1 Idea básica

Un GMM modela cada grupo como una distribución gaussiana multivariada.

En dos dimensiones, podemos imaginar cada componente como una nube o una elipse. Cada componente tiene su propia media, su propia dispersión y, según el modelo, su propia forma.

La ventaja frente a k-means es que GMM no devuelve solo una etiqueta de grupo. También devuelve probabilidades de pertenencia. Eso permite mirar qué observaciones están claramente asignadas y cuáles están en zonas dudosas.

1.2 Ejemplo con `faithful`

El dataset `faithful` tiene datos del géiser Old Faithful. Usa dos variables:

- `eruptions`: duración de la erupción.
- `waiting`: tiempo de espera hasta la próxima erupción.

```
data(faithful)
```

```
dim(faithful)
```

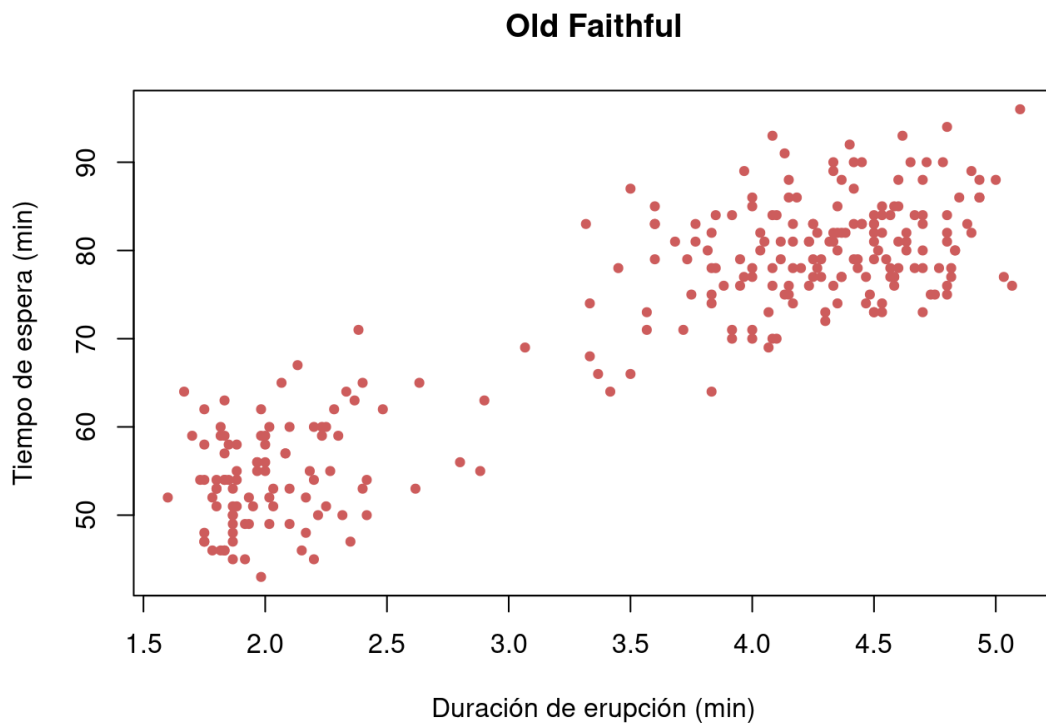
```
[1] 272 2
```

```
head(faithful)
```

```
eruptions waiting
1      3.600      79
2      1.800     54
```

2	1.8000	54
3	3.3333	74
4	2.2833	62
5	4.5333	85
6	2.8833	55

```
plot(faithful,
     pch = 19,
     cex = 0.7,
     col = "indianred",
     xlab = "Duración de erupción (min)",
     ylab = "Tiempo de espera (min)",
     main = "Old Faithful")
```



Visualmente se ven zonas de puntos. GMM intenta modelar esas zonas como componentes gaussianos.

```
gmm_faithful <- Mclust(faithful)
summary(gmm_faithful)
```

```
-----
Gaussian finite mixture model fitted by EM algorithm
-----
```

```
Mclust EEE (ellipsoidal, equal volume, shape and orientation) model
with 3
```

components:

log-likelihood	n	df	BIC	ICL
-1126.326	272	11	-2314.316	-2357.824

Clustering table:

1	2	3
40	97	135

El resumen muestra el modelo elegido, la cantidad de componentes y el tamaño de cada grupo.

En GMM se habla de componentes. En la práctica, para clustering, esos componentes son los grupos gaussianos estimados.

1.3 BIC

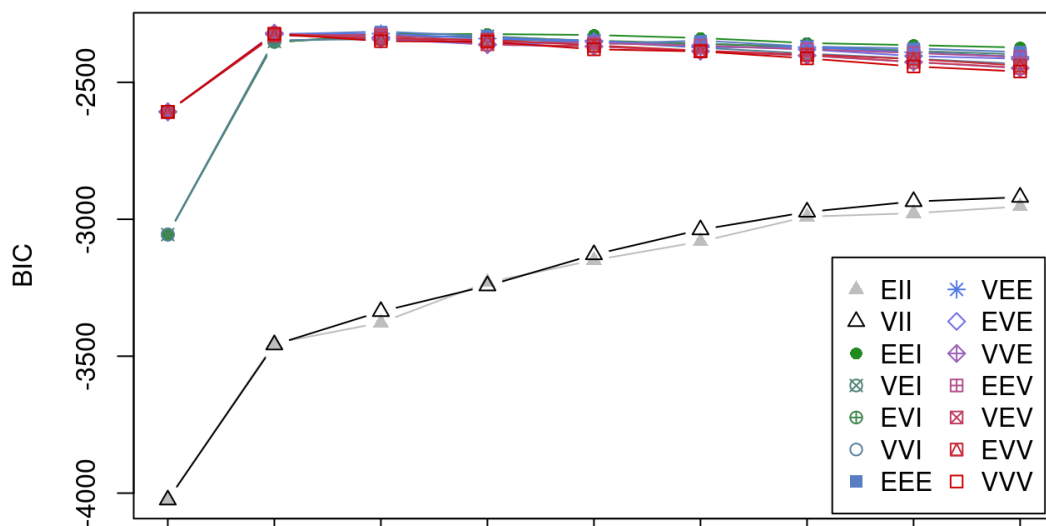
`Mclust()` prueba varios modelos y varias cantidades de componentes. Luego selecciona usando BIC.

La regla es simple:

mayor BIC = mejor modelo dentro de los comparados

Los BIC pueden ser negativos. Eso no importa. Si son negativos, el mayor es el que está más cerca de cero.

```
plot(gmm_faithful, what = "BIC")
```



1 2 3 4 5 6 7 8 9

Number of components

`gmm_faithful$BIC`

Bayesian Information Criterion (BIC):

	EII	VII	EEI	VEI	EVI	VVI	EEE
1	-4024.721	-4024.721	-3055.835	-3055.835	-3055.835	-3055.835	-2607.623
2	-3452.998	-3458.305	-2354.601	-2350.607	-2352.618	-2346.065	-2325.220
3	-3377.701	-3336.598	-2323.014	-2332.687	-2332.205	-2342.366	-2314.316
4	-3230.264	-3242.826	-2323.673	-2331.284	-2334.749	-2343.486	-2331.223
5	-3149.394	-3129.080	-2327.059	-2350.230	-2347.564	-2351.017	-2360.659
6	-3081.414	-3038.171	-2338.205	-2360.578	-2357.660	-2373.469	-2347.352
7	-2990.367	-2973.374	-2356.454	-2368.513	-2372.851	-2394.696	-2369.330
8	-2978.100	-2935.082	-2364.140	-2384.740	-2389.064	-2413.705	-2376.104
9	-2953.359	-2919.415	-2372.790	-2398.223	-2407.224	-2432.708	-2389.609
	VEE	EVE	VVE	EEV	VEV	EVV	VVV
1	-2607.623	-2607.623	-2607.623	-2607.623	-2607.623	-2607.623	-2607.623
2	-2322.972	-2324.273	-2320.433	-2329.115	-2325.416	-2327.598	-2322.192
3	-2322.103	-2342.319	-2336.271	-2325.322	-2329.648	-2339.983	-2349.696
4	-2340.173	-2361.821	-2362.487	-2351.523	-2361.084	-2344.686	-2351.493
5	-2347.337	-2351.828	-2368.937	-2356.856	-2368.101	-2364.900	-2379.388
6	-2372.287	-2366.482	-2386.537	-2366.087	-2386.323	-2384.117	-2387.016
7	-2371.175	-2379.810	-2402.220	-2379.071	-2401.270	-2398.703	-2412.440
8	-2390.391	-2403.934	-2425.956	-2392.988	-2425.426	-2414.962	-2442.018
9	-2406.732	-2414.089	-2448.208	-2407.500	-2446.726	-2438.876	-2460.398

Top 3 models based on the BIC criterion:

EEE,3 VVE,2 VEE,3
-2314.316 -2320.433 -2322.103

Las filas representan la cantidad de componentes y las columnas representan distintos modelos de forma/covarianza.

Nombres como **EEE**, **VEV** o **VVV** describen la forma de las nubes gaussianas.

1ª letra -> volumen / tamaño

2ª letra -> forma

3ª letra -> orientación

E = igual entre grupos

V = variable entre grupos

I = esférico / sin orientación especial

Por ejemplo:

EEE -> grupos elipsoidales con tamaño, forma y orientación iguales
 VVV -> grupos elipsoidales con tamaño, forma y orientación variables
 EII -> grupos esféricos de igual tamaño

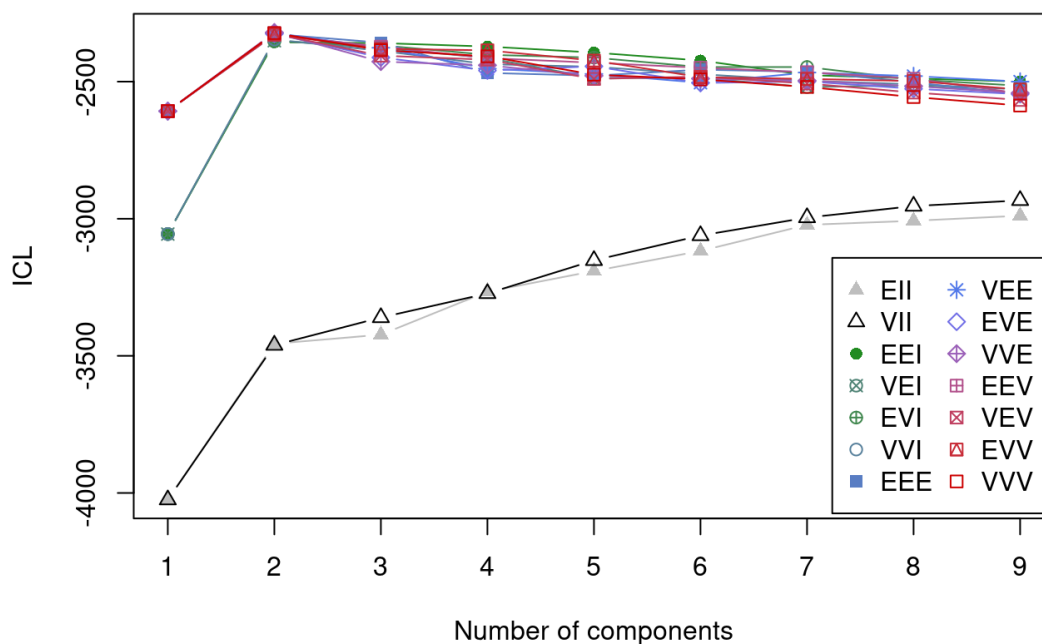
1.4 ICL

ICL es parecido a BIC, pero penaliza más la incertidumbre de clasificación.

Una forma simple de pensarlo:

BIC -> buen ajuste, sin demasiada complejidad
 ICL -> buen ajuste, pero además grupos más claros

```
icl_faithful <- mclustICL(faithful)
plot(icl_faithful)
```



```
summary(icl_faithful)
```

Best ICL values:

	VVE, 2	VVV, 2	VEE, 2
ICL	-2320.763	-2322.697467	-2323.39551
ICL diff	0.000	-1.934645	-2.63269

Si BIC e ICL eligen modelos distintos, no significa que uno esté mal. Están priorizando cosas distintas.

En este ejemplo podemos ajustar el modelo sugerido por ICL para compararlo.

```
gmm_faithful_icl <- Mclust(faithful,  
                           G = 2,  
                           modelNames = "VVE")  
summary(gmm_faithful_icl)
```

```
-----  
Gaussian finite mixture model fitted by EM algorithm  
-----
```

Mclust VVE (ellipsoidal, equal orientation) model with 2 components:

log-likelihood	n	df	BIC	ICL
-1132.187	272	10	-2320.433	-2320.763

Clustering table:

1	2
175	97

1.5 Clasificación, probabilidades e incertidumbre

La parte más importante de GMM para aprender es esta:

classification -> grupo más probable
z -> probabilidades de pertenencia
uncertainty -> duda del modelo

```
head(gmm_faithful$z)
```

	[,1]	[,2]	[,3]
1	9.718036e-01	1.420275e-08	2.819634e-02
2	8.604351e-13	1.000000e+00	7.571495e-21
3	9.967399e-01	2.175555e-05	3.238336e-03
4	3.140299e-07	9.999997e-01	1.494469e-13
5	1.168409e-02	1.302547e-19	9.883159e-01
6	3.092711e-03	9.969070e-01	3.390998e-07

Cada fila corresponde a una observación y cada columna a un componente.
Cada fila suma 1.

Si una fila tiene una probabilidad cercana a 1 en una columna, la asignación es clara. Si las probabilidades están repartidas, el modelo duda.

```
head(gmm_faithful$classification)
```

```
1 2 3 4 5 6  
1 2 1 2 3 2
```

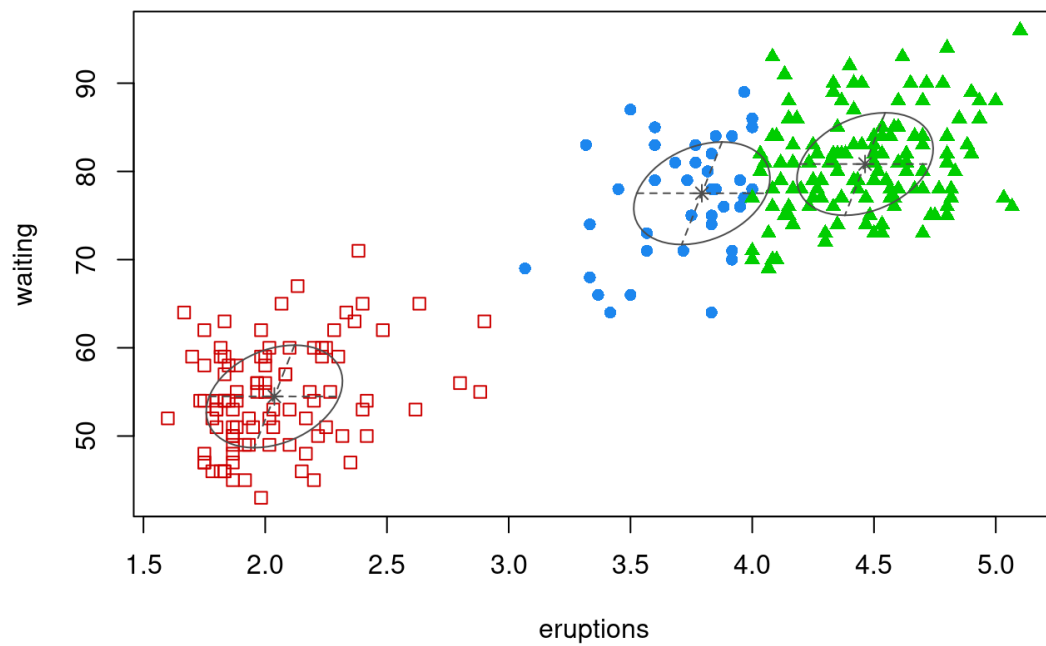
```
table(gmm_faithful$classification)
```

```
1 2 3  
40 97 135
```

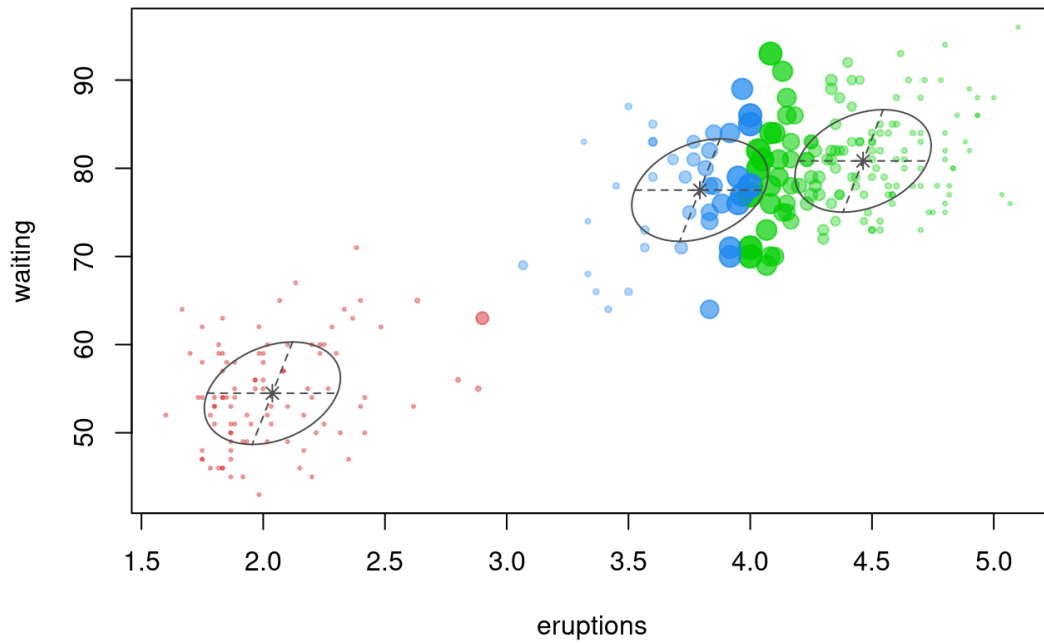
```
summary(gmm_faithful$uncertainty)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.000000	0.000000	0.004697	0.067098	0.061553	0.497355

```
plot(gmm_faithful, what = "classification")
```



```
plot(gmm_faithful, what = "uncertainty")
```



1.6 Certeza de asignación

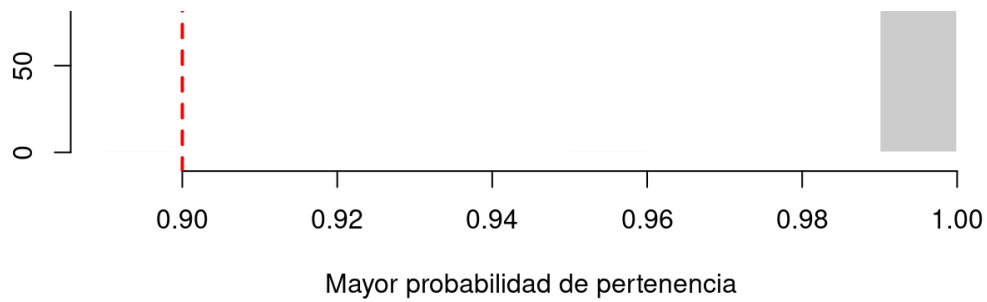
Una forma muy directa de mirar la certeza es tomar, para cada observación, su mayor probabilidad de pertenencia.

```
prob_max <- apply(gmm_faithful_icl$z, 1, max)

hist(prob_max,
     main = "Certeza de asignación",
     xlab = "Mayor probabilidad de pertenencia",
     col = "gray80",
     border = "white")
abline(v = 0.9, col = "red", lwd = 2, lty = 2)
```

Certeza de asignación





Valores cercanos a 1 indican asignaciones claras. Valores más bajos indican observaciones ubicadas en zonas de solapamiento.

1.7 Observaciones dudosas

También podemos buscar las observaciones donde el modelo tiene más dudas.

```
obs_dudosas <- head(order(prob_max), 10)

faithful[obs_dudosas, ]
```

	eruptions	waiting
24	3.067	69
244	2.900	63
6	2.883	55
84	2.633	65
174	3.333	68
33	3.367	66
211	2.383	71
133	2.800	56
215	3.417	64
3	3.333	74

```
gmm_faithful_icl$z[obs_dudosas, ]
```

	[,1]	[,2]
24	0.8967047326	0.1032952674
244	0.0468262073	0.9531737927
6	0.0021852494	0.9978147506
84	0.0014002258	0.9985997742
174	0.9989783254	0.0010216746
33	0.9990483485	0.0009516515
211	0.0007455476	0.9992544524
133	0.0006740231	0.9993259769
215	0.9994030629	0.0005969371
3	0.9998637077	0.0001362923

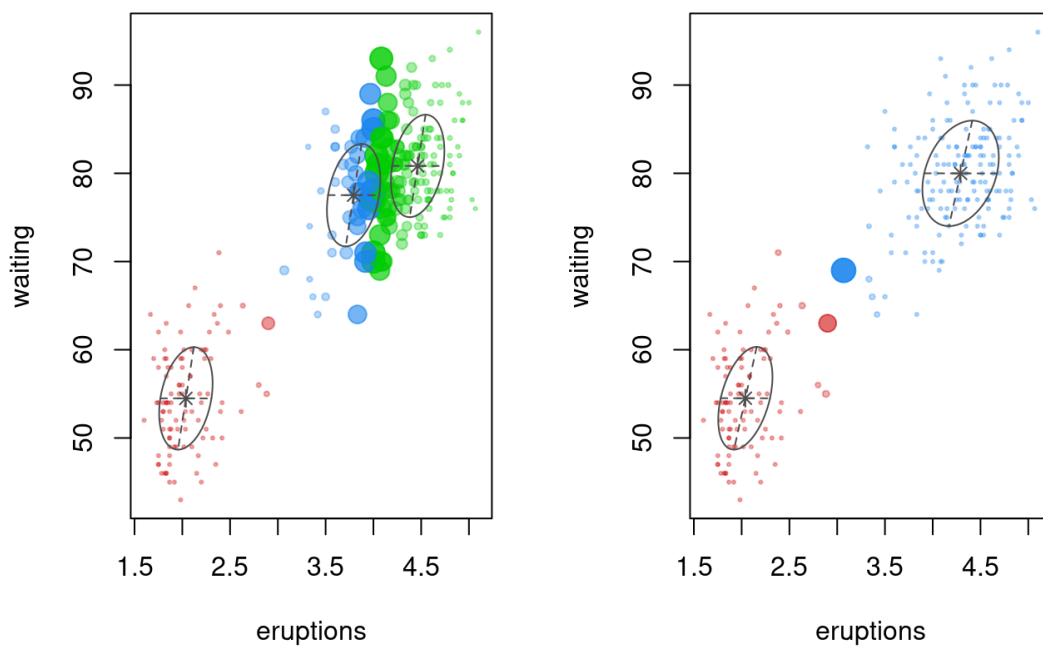
```
gmm_faithful_icl$classification[obs_dudosas]
```

```
24 244    6   84 174   33 211 133 215    3
  1   2    2    2    1    1   2   2    1    1
```

Estas observaciones no necesariamente están mal clasificadas. Simplemente están en zonas donde dos componentes se solapan o donde el modelo no tiene una asignación clara.

1.8 Comparación BIC vs ICL

```
par(mfrow = c(1, 2))
plot(gmm_faithful, what = "uncertainty")
plot(gmm_faithful_icl, what = "uncertainty")
```



```
par(mfrow = c(1, 1))
```

ICL suele favorecer soluciones con menor solapamiento. Por eso puede mostrar menor incertidumbre, aunque use menos componentes.

2. GMM con Iris

Con `iris` tenemos una ventaja: conocemos la especie real de cada flor.

Esto permite preguntar:

¿Los grupos estimados por GMM se parecen a las especies reales?

```
data(iris)
X_iris <- iris[, 1:4]

gmm_iris <- Mclust(X_iris)
summary(gmm_iris)
```

Gaussian finite mixture model fitted by EM algorithm

Mclust VEV (ellipsoidal, equal shape) model with 2 components:

log-likelihood	n	df	BIC	ICL
-215.726	150	26	-561.7285	-561.7289

Clustering table:

1	2
50	100

```
table(GMM = gmm_iris$classification,
      Species = iris$Species)
```

	Species		
GMM	setosa	versicolor	virginica
1	50	0	0
2	0	50	50

En este caso, el modelo automático puede resumir los datos en 2 componentes: uno separa `setosa`, y el otro agrupa `versicolor` + `virginica`.

Eso no significa que Iris “tenga dos especies”. Significa que, según el criterio del modelo, la estructura gaussiana de las variables se resume mejor así.

2.1 Forzar G = 3

Como sabemos que Iris tiene 3 especies, podemos forzar un modelo con 3 componentes para comparar.

```
gmm_iris3 <- Mclust(X_iris, G = 3)
summary(gmm_iris3)
```

 Gaussian finite mixture model fitted by EM algorithm

Mclust VEV (ellipsoidal, equal shape) model with 3 components:

log-likelihood	n	df	BIC	ICL
-186.074	150	38	-562.5522	-566.4673

Clustering table:

1	2	3
50	45	55

```
table(GMM = gmm_iris3$classification,
      Species = iris$Species)
```

	Species		
GMM	setosa	versicolor	virginica
1	50	0	0
2	0	45	0
3	0	5	50

Esta comparación permite ver si los componentes estimados coinciden con las especies reales.

2.2 MclustDR

MclustDR() no busca nuevos grupos. Toma un GMM ya ajustado y construye una proyección donde esos componentes se vean mejor separados.

Diferencia con PCA:

PCA -> busca ejes de máxima variación total

MclustDR -> busca ejes que separen mejor los componentes del GMM

```
dr_iris_2 <- MclustDR(gmm_iris)
summary(dr_iris_2)
```

 Dimension reduction for model-based clustering and classification

Mixture model type: Mclust (VEV 2)

```
mixture_model_type: mclust (vzv, 2)
```

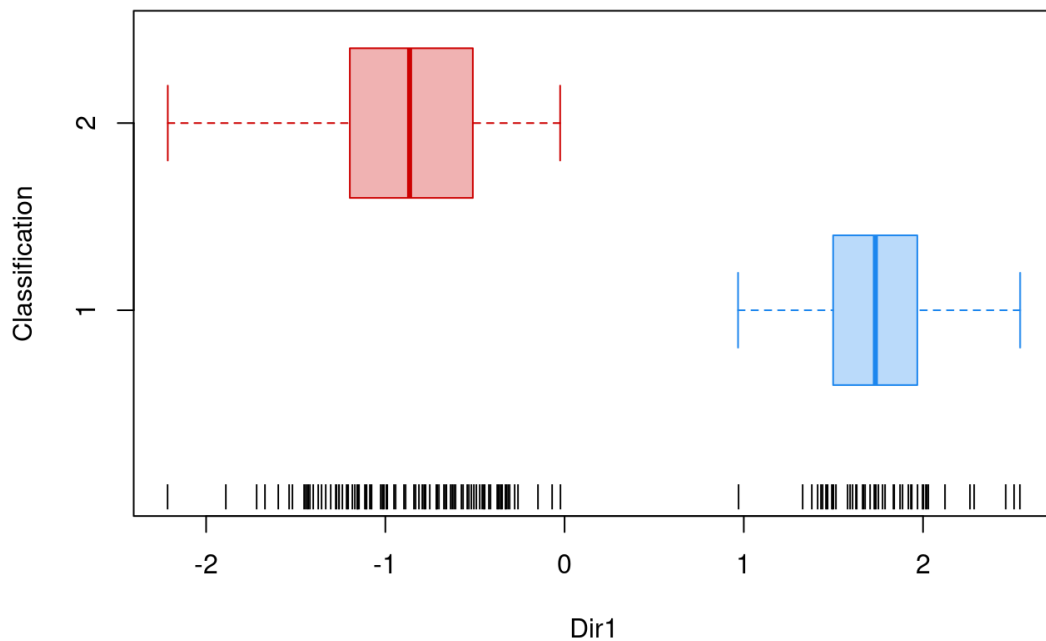
```
Clusters      n
      1      50
      2     100
```

Estimated basis vectors:

```
          Dir1
Sepal.Length  0.19293
Sepal.Width   0.70967
Petal.Length -0.65645
Petal.Width  -0.16798
```

```
          Dir1
Eigenvalues  1.6489
Cum. %      100.0000
```

```
plot(dr_iris_2, what = "boundaries")
```



Con 2 componentes, alcanza una sola dirección para separar *setosa* del resto.

```
dr_iris_3 <- MclustDR(gmm_iris3)
summary(dr_iris_3)
```

Dimension reduction for model-based clustering and classification

Mixture model type: Mclust (VEV, 3)

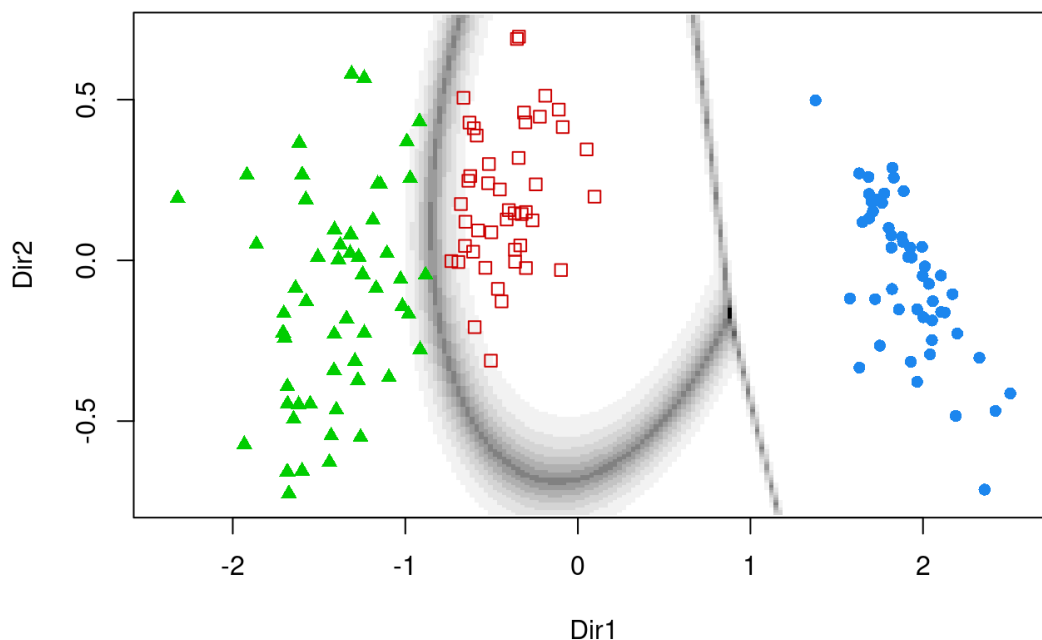
```
Clusters  n
      1  50
      2  45
      3  55
```

Estimated basis vectors:

	Dir1	Dir2
Sepal.Length	0.20399	-0.013915
Sepal.Width	0.43511	-0.556598
Petal.Length	-0.55859	0.264589
Petal.Width	-0.67606	-0.787399

	Dir1	Dir2
Eigenvalues	1.8786	0.072507
Cum. %	96.2837	100.000000

```
plot(dr_iris_3, what = "boundaries")
```



Con 3 componentes, la proyección muestra una estructura más cercana a las tres especies. Aun así, este modelo fue forzado: no fue necesariamente el preferido por BIC.

2.3 Comparar GMM con k-means

Ahora comparamos si GMM y k-means agrupan las flores de forma parecida.

```
set.seed(42)
km_iris <- kmeans(scale(X_iris), centers = 3, nstart = 25)

tabla_conc <- table(GMM = gmm_iris3$classification,
                    Kmeans = km_iris$cluster)
tabla_conc
```

```
      Kmeans
GMM  1  2  3
  1 50  0  0
  2  0 36  9
  3  0 17 38
```

La tabla muestra cómo se cruzan las dos particiones.

```
ari_gmm_km <- adjustedRandIndex(gmm_iris3$classification,
                                km_iris$cluster)
ari_gmm_km
```

```
[1] 0.6108795
```

El ARI mide concordancia entre particiones.

ARI = 1 -> coincidencia perfecta
ARI = 0 -> coincidencia similar al azar

3. DBSCAN — grupos por densidad

3.1 Idea básica

DBSCAN cambia la lógica. No busca centroides ni gaussianas.

Busca zonas densas conectadas.

Tiene dos parámetros principales:

eps -> radio de vecindad
minPts -> cantidad mínima de puntos dentro de ese radio

Un punto núcleo es un punto que tiene al menos `minPts` puntos dentro del radio

`eps`.

Los clusters se forman conectando puntos núcleo y puntos de borde. Los puntos que no quedan conectados a ninguna zona densa se etiquetan como ruido.

En `dbscan()`, el ruido aparece como cluster 0.

La analogía que usamos:

puntos = átomos
eps = volumen local que miro
minPts = cantidad mínima de átomos para formar cuerpo
zona densa conectada = grupo / cuerpo
puntos aislados = ruido / aire

3.2 Datos simulados: anillos y medias lunas

Usamos formas donde k-means suele fallar.

```
n <- 200
theta <- seq(0, 2*pi, length.out = n)

anillo1 <- cbind(cos(theta) * 1 + rnorm(n, 0, 0.08),
                 sin(theta) * 1 + rnorm(n, 0, 0.08))
anillo2 <- cbind(cos(theta) * 2.5 + rnorm(n, 0, 0.08),
                 sin(theta) * 2.5 + rnorm(n, 0, 0.08))

luna1 <- cbind(cos(seq(0, pi, length.out = n)) * 1.5 + rnorm(n, 0, 0.08),
               sin(seq(0, pi, length.out = n)) * 1 + rnorm(n, 0, 0.08))
luna2 <- cbind(cos(seq(pi, 2*pi, length.out = n)) * 1.5 + rnorm(n, 0, 0.08),
               sin(seq(pi, 2*pi, length.out = n)) * 1 + rnorm(n, 0, 0.08))

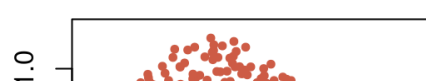
anillos <- rbind(anillo1, anillo2)
lunas <- rbind(luna1, luna2)
```

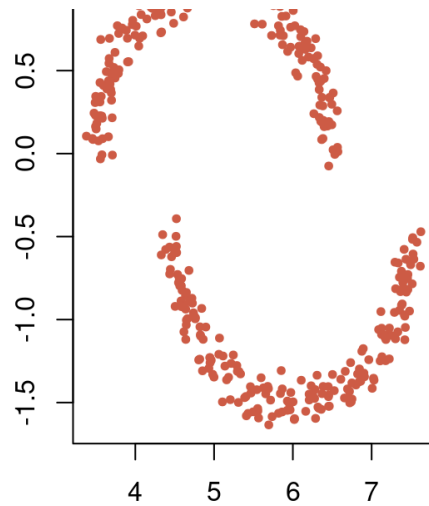
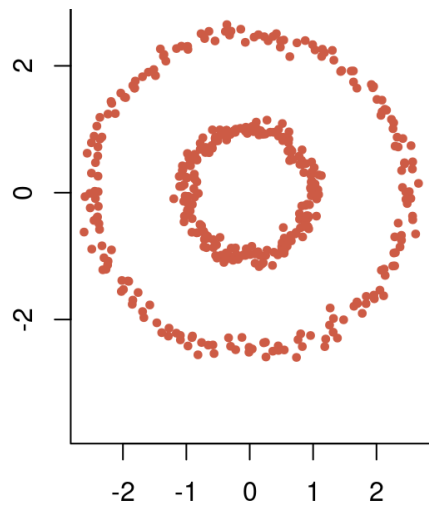
```
par(mfrow = c(1, 2))
plot(anillos, pch = 19, cex = 0.6, col = "coral3", asp = 1,
     main = "Anillos", xlab = "", ylab = "")
plot(lunas, pch = 19, cex = 0.6, col = "coral3",
     main = "Medias lunas", xlab = "", ylab = "")
```

Anillos



Medias lunas





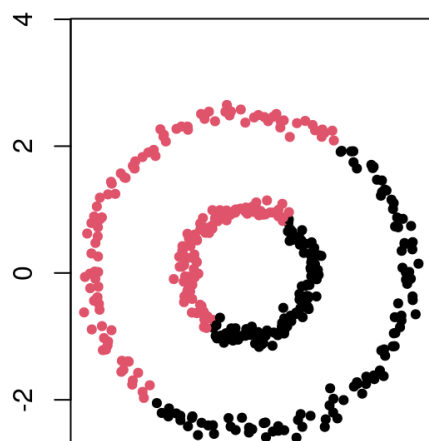
```
par(mfrow = c(1, 1))
```

3.3 K-means como contraste

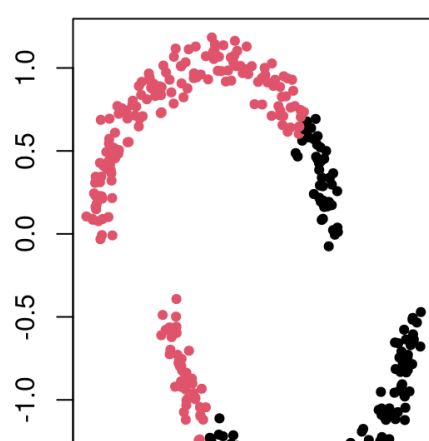
```
set.seed(42)
km_anillos <- kmeans(anillos, centers = 2, nstart = 25)
km_lunas <- kmeans(lunas, centers = 2, nstart = 25)

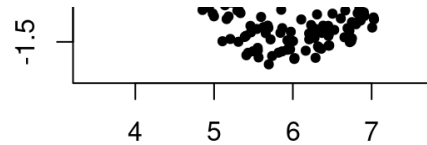
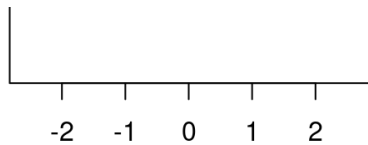
par(mfrow = c(1, 2))
plot(anillos, col = km_anillos$cluster, pch = 19, cex = 0.7, asp = 1,
     main = "K-means en anillos", xlab = "", ylab = "")
plot(lunas, col = km_lunas$cluster, pch = 19, cex = 0.7,
     main = "K-means en medias lunas", xlab = "", ylab = "")
```

K-means en anillos



K-means en medias lunas





```
par(mfrow = c(1, 1))
```

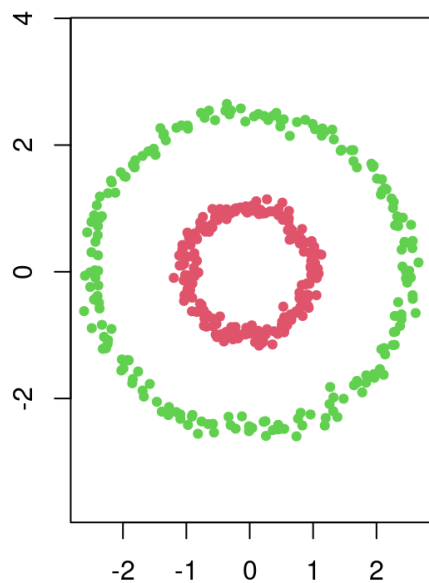
K-means parte la estructura porque busca grupos alrededor de centroides.

3.4 DBSCAN

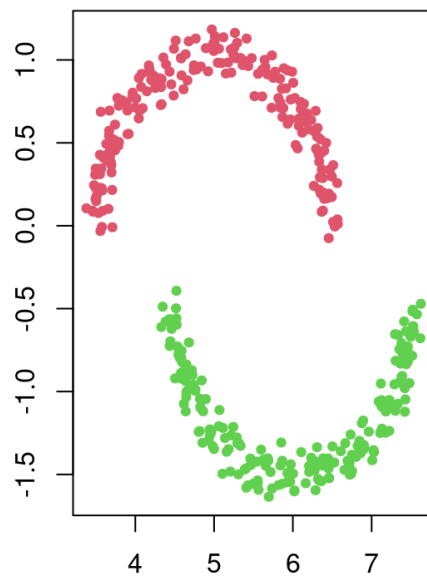
```
db_anillos <- dbscan(anillos, eps = 0.40, minPts = 4)
db_lunas <- dbscan(lunas, eps = 0.20, minPts = 4)
```

```
par(mfrow = c(1, 2))
plot(anillos, col = db_anillos$cluster + 1, pch = 19, cex = 0.7, asp = 1,
     main = "DBSCAN en anillos", xlab = "", ylab = "")
plot(lunas, col = db_lunas$cluster + 1, pch = 19, cex = 0.7,
     main = "DBSCAN en medias lunas", xlab = "", ylab = "")
```

DBSCAN en anillos



DBSCAN en medias lunas



```
par(mfrow = c(1, 1))
```

```
table(db_anillos$cluster)
```

```
1 2  
200 200
```

```
table(db_lunas$cluster)
```

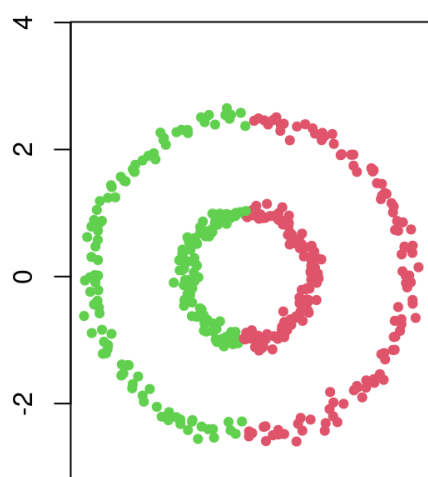
```
1 2  
200 200
```

Los valores de `eps` no son universales. Dependen de la escala y densidad de los datos. En esta primera lectura conviene ajustarlos manualmente para ver el efecto conceptual.

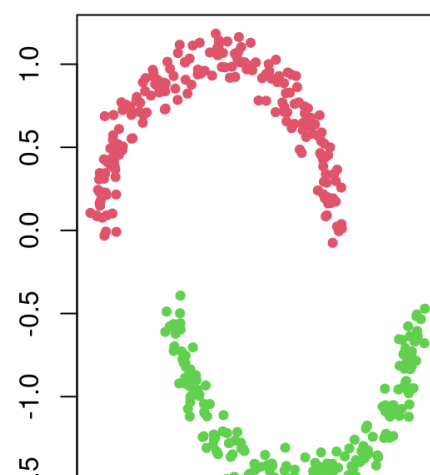
3.5 Comparación con GMM

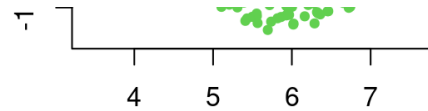
```
gmm_anillos <- Mclust(anillos, G = 2)  
gmm_lunas <- Mclust(lunas, G = 2)  
  
par(mfrow = c(1, 2))  
plot(anillos, col = gmm_anillos$classification + 1, pch = 19, cex = 0.7,  
      main = "GMM G = 2 — Anillos", xlab = "", ylab = "")  
plot(lunas, col = gmm_lunas$classification + 1, pch = 19, cex = 0.7,  
      main = "GMM G = 2 — Medias lunas", xlab = "", ylab = "")
```

GMM G = 2 — Anillos



GMM G = 2 — Medias lunas





```
par(mfrow = c(1, 1))
```

En los anillos, GMM puede fallar de forma parecida a k-means: corta la estructura en regiones, pero no recupera anillo interno vs. anillo externo.

En las medias lunas, GMM puede funcionar bien. En este ejemplo, incluso resulta más simple que DBSCAN porque solo indicamos $G = 2$. Esto muestra que no hay un método mejor en abstracto. Cada método trae una idea distinta de qué es un grupo.

3.6 Puntos núcleo

Esta parte desarma DBSCAN por dentro.

Un punto es núcleo si tiene al menos minPts vecinos dentro del radio eps .

```
eps_anillos <- 0.40
minpts_anillos <- 4

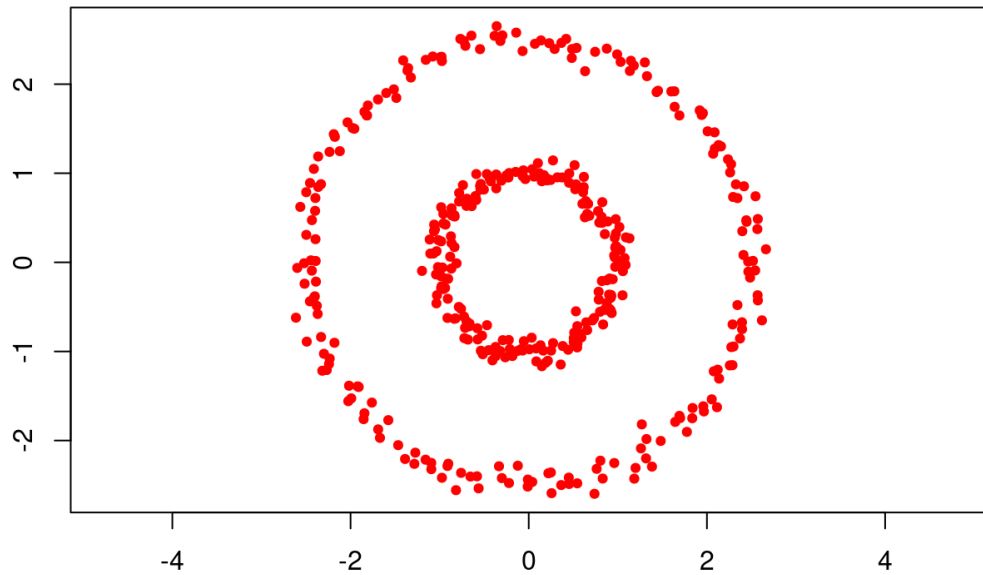
nn_anillos <- kNN(anillos, k = minpts_anillos)
es_nucleo <- nn_anillos$dist[, minpts_anillos] <= eps_anillos

sum(es_nucleo)
```

```
[1] 400
```

```
plot(anillos,
     col = ifelse(es_nucleo, "red", "gray70"),
     pch = 19,
     cex = 0.7,
     asp = 1,
     main = "Puntos núcleo en anillos",
     xlab = "",
     ylab = "")
```

Puntos núcleo en anillos



Esto no es necesario para correr DBSCAN, pero ayuda a entender cómo decide dónde hay densidad.

4. OPTICS

4.1 Idea básica

OPTICS es una extensión de DBSCAN.

DBSCAN usa un único **eps**. Eso puede ser un problema cuando hay zonas de distinta densidad.

OPTICS no devuelve primero una partición fija. Construye un ordenamiento de los puntos y un gráfico de alcanzabilidad.

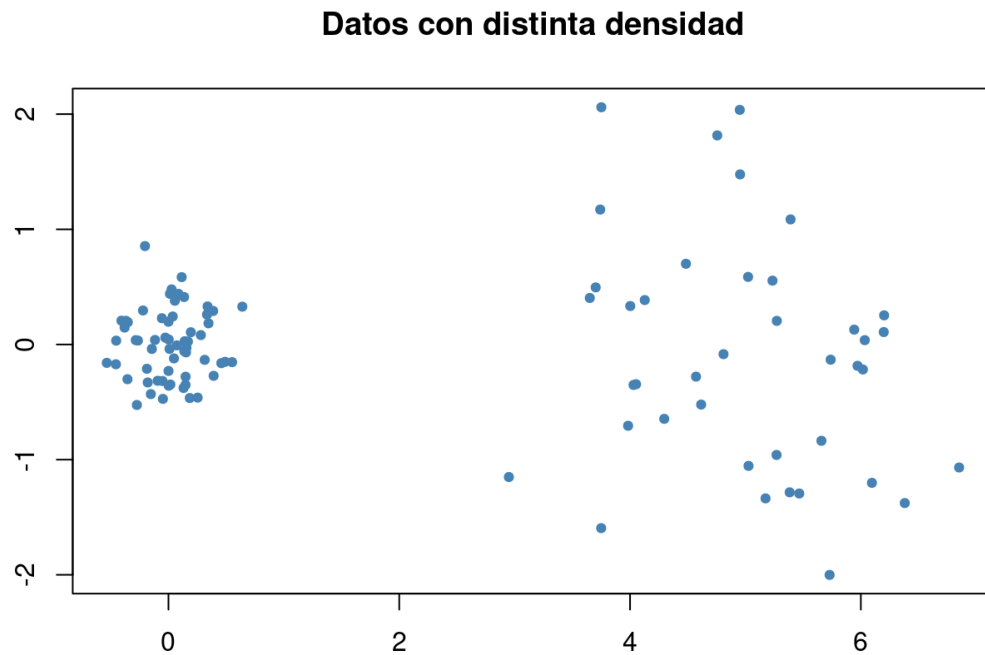
En el reachability plot:

barras bajas -> puntos fáciles de alcanzar -> zonas densas
barras altas -> puntos difíciles de alcanzar -> zonas dispersas o separación
valles -> posibles clusters
picos -> separación o ruido

No es un gráfico de densidad directa. Es un gráfico de distancia de alcanzabilidad. Por eso lo denso aparece abajo.

4.2 Datos con distinta densidad

```
X_dens <- rbind(  
  mvrnorm(60, mu = c(0, 0), Sigma = diag(0.1, 2)),  
  mvrnorm(40, mu = c(5, 0), Sigma = diag(1.0, 2))  
)  
  
plot(X_dens,  
     pch = 19,  
     cex = 0.7,  
     col = "steelblue",  
     main = "Datos con distinta densidad",  
     xlab = "",  
     ylab = "")
```



Un grupo es denso y el otro es más disperso.

4.3 Ajuste OPTICS

```
opt_dens <- optics(X_dens,  
                  eps = Inf,  
                  minPts = 5)  
  
opt_dens
```

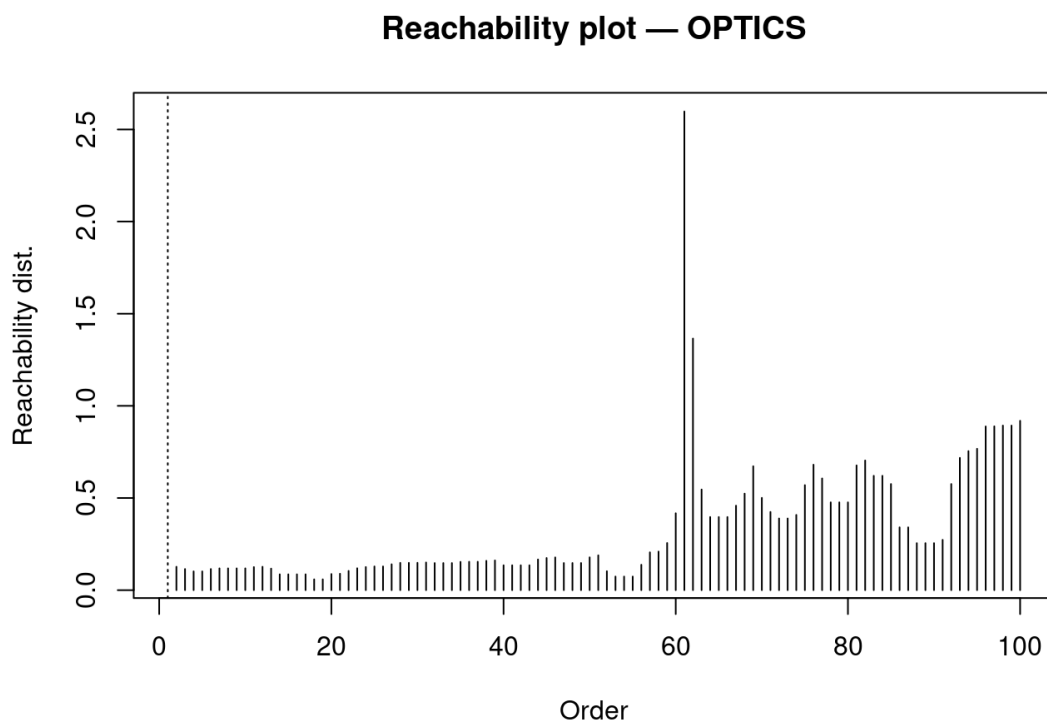
OPTICS ordering/clustering for 100 objects.

Parameters: minPts = 5, eps = Inf, eps_cl = NA, xi = NA

Available fields: order, reachdist, coredist, predecessor, minPts, eps, eps_cl, xi

En OPTICS suele usarse `eps = Inf` para no limitar el radio inicial. El parámetro clave pasa a ser `minPts`.

```
plot(opt_dens,  
     main = "Reachability plot — OPTICS")
```



El grupo denso aparece como un valle bajo. El grupo disperso aparece como un valle más alto. La separación aparece como un pico.

4.4 Extraer clusters desde OPTICS

OPTICS no devuelve una única partición final. Para convertir el reachability plot en grupos concretos podemos usar `extractDBSCAN()`.

`eps_cl` funciona como una línea horizontal de corte.

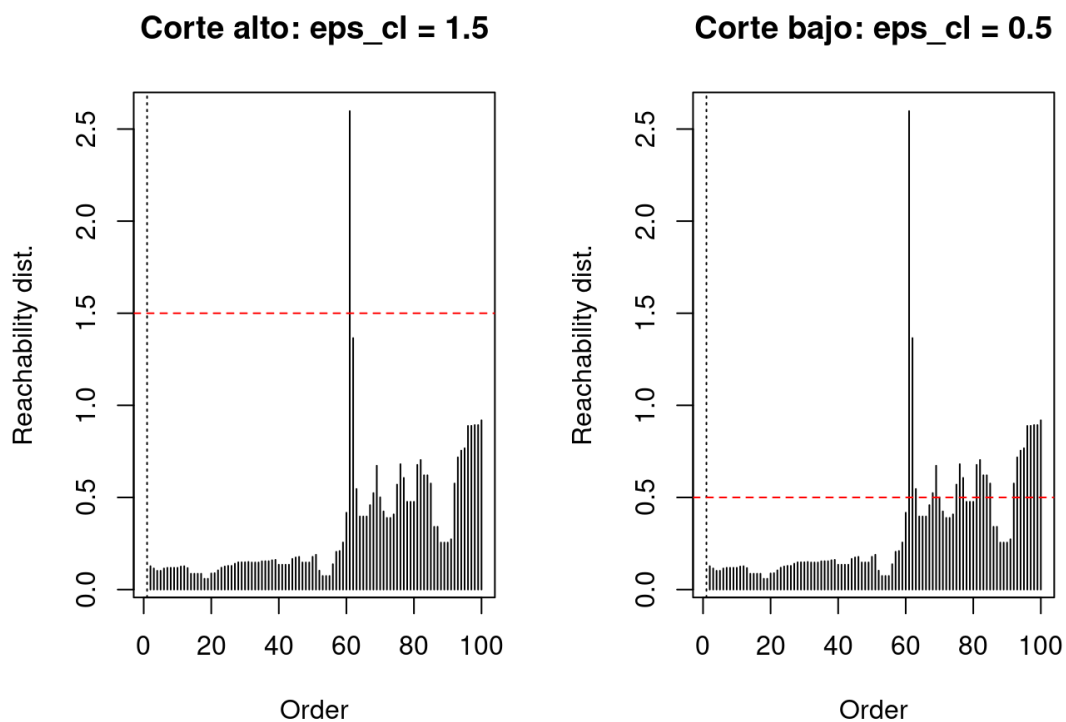
`eps_cl` alto -> corte permisivo, permite grupos menos densos

`eps_cl` bajo -> corte exigente, conserva sobre todo zonas muy densas

```
opt_alto <- extractDBSCAN(opt_dens, eps_cl = 1.5)
opt_bajo <- extractDBSCAN(opt_dens, eps_cl = 0.5)
```

```
par(mfrow = c(1, 2))
plot(opt_dens, main = "Corte alto: eps_cl = 1.5")
abline(h = 1.5, col = "red", lty = 2)

plot(opt_dens, main = "Corte bajo: eps_cl = 0.5")
abline(h = 0.5, col = "red", lty = 2)
```



```
par(mfrow = c(1, 1))
```

```
table(opt_alto$cluster)
```

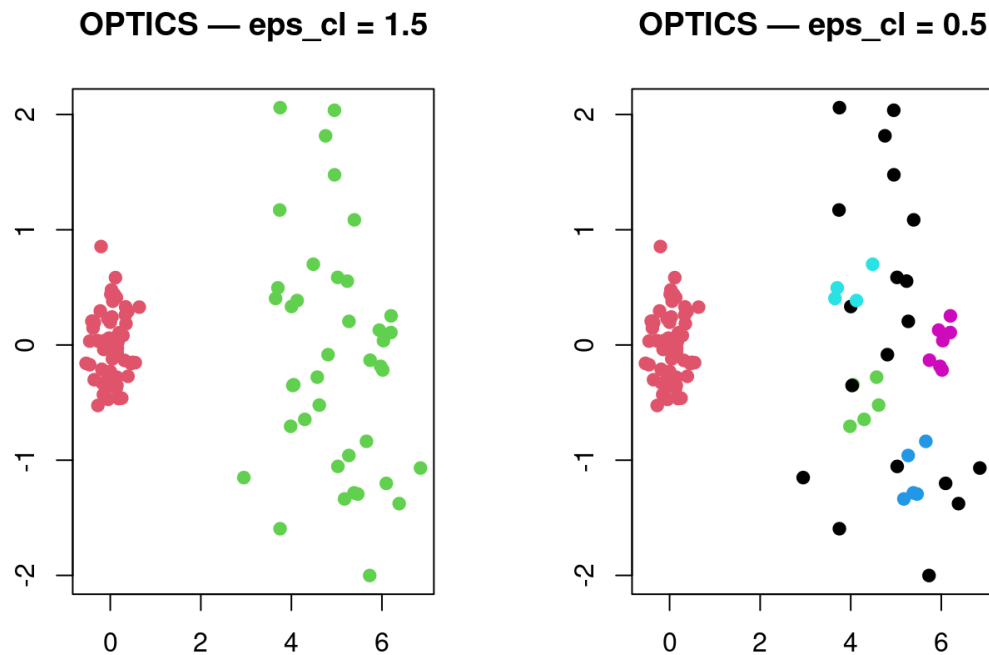
```
1 2
60 40
```

```
table(opt_bajo$cluster)
```

```
0 1 2 3 4 5
```

19 60 5 5 4 7

```
par(mfrow = c(1, 2))
plot(X_dens, col = opt_alto$cluster + 1, pch = 19,
     main = "OPTICS — eps_cl = 1.5", xlab = "", ylab = "")
plot(X_dens, col = opt_bajo$cluster + 1, pch = 19,
     main = "OPTICS — eps_cl = 0.5", xlab = "", ylab = "")
```



```
par(mfrow = c(1, 1))
```

Con `eps_cl = 1.5`, el corte es más permisivo y puede recuperar tanto el grupo denso como el grupo disperso.

Con `eps_cl = 0.5`, el corte es más exigente. Mantiene las zonas muy densas, pero puede dejar como ruido parte de la zona dispersa.

4.5 Extracción automática con `xi`

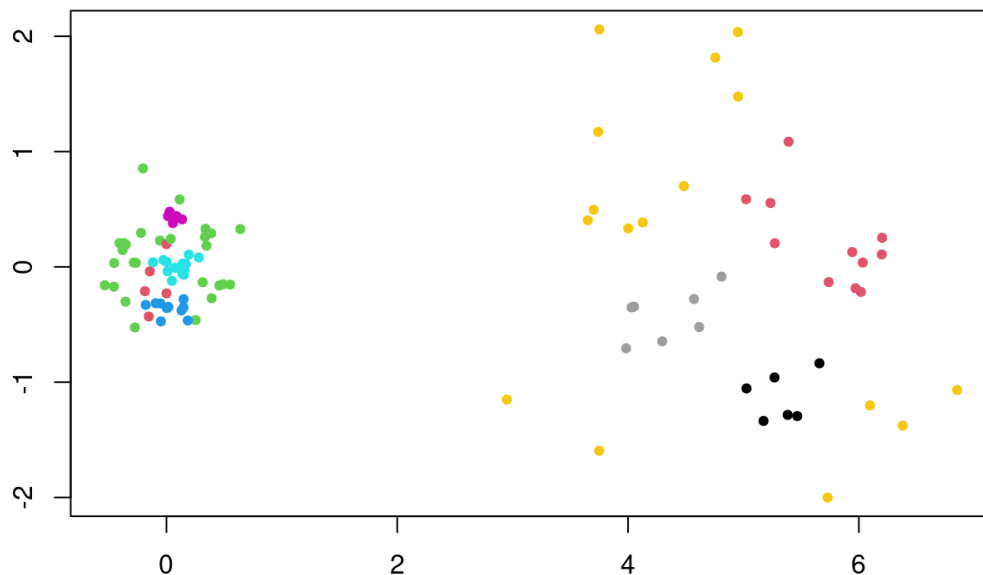
`extractXi()` intenta extraer clusters automáticamente buscando cambios relativos de densidad.

```
opt_xi <- extractXi(opt_dens, xi = 0.05)
table(opt_xi$cluster)
```

```
1 2 3 4 5 6 7 8 9
5 27 10 13 5 16 7 6 11
```

```
plot(X_dens,
     col = opt_xi$cluster + 1,
     pch = 19,
     cex = 0.7,
     main = "OPTICS — extracción automática xi",
     xlab = "",
     ylab = "")
```

OPTICS — extracción automática xi



Para una primera lectura, `eps_cl` es más fácil de entender porque se ve como un corte horizontal. `xi` puede quedar como opción automática complementaria.

5. Complementario: crabs

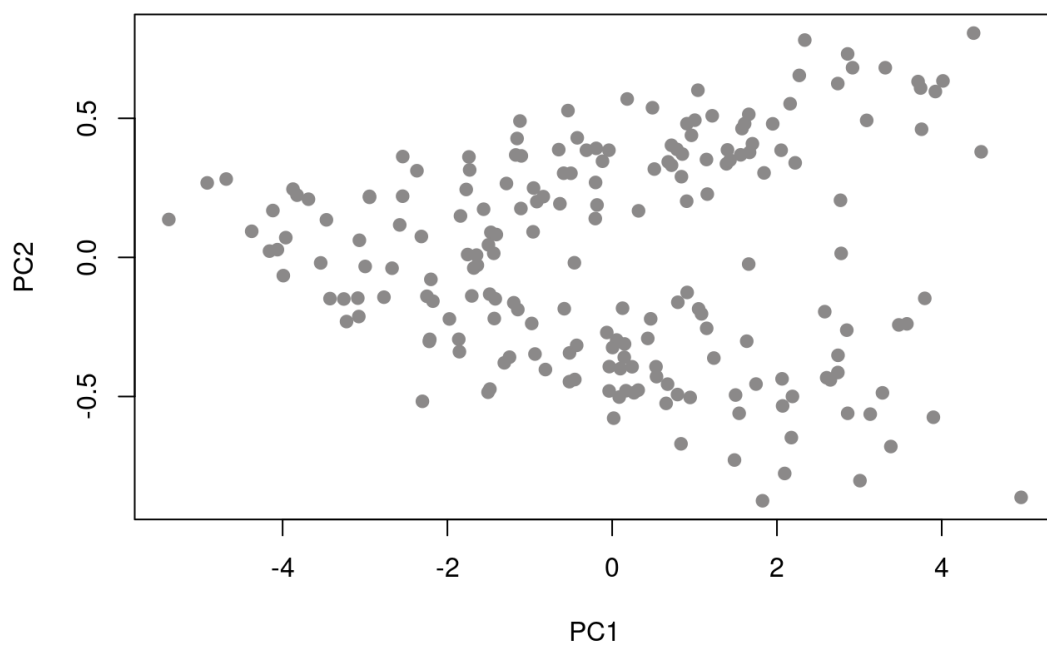
Con datos reales, DBSCAN y OPTICS requieren más decisiones: selección de variables, escalado, valores de `eps` y `minPts`, y una interpretación más cuidadosa.

Por eso esta sección queda como exploración complementaria.

```
data(crabs)
X_crabs <- scale(crabs[, 4:8])
X_crabs_pca <- prcomp(X_crabs)$x[, 1:2]

plot(X_crabs_pca,
     pch = 19,
     col = "snow4",
     main = "Crabs — proyección PCA",
     xlab = "PC1",
     ylab = "PC2")
```

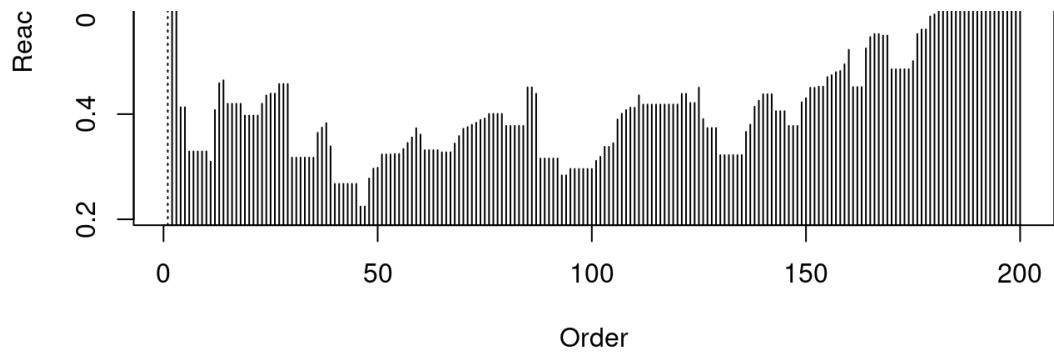
Crabs — proyección PCA



```
opt_crabs <- optics(X_crabs, eps = Inf, minPts = 8)
plot(opt_crabs)
```

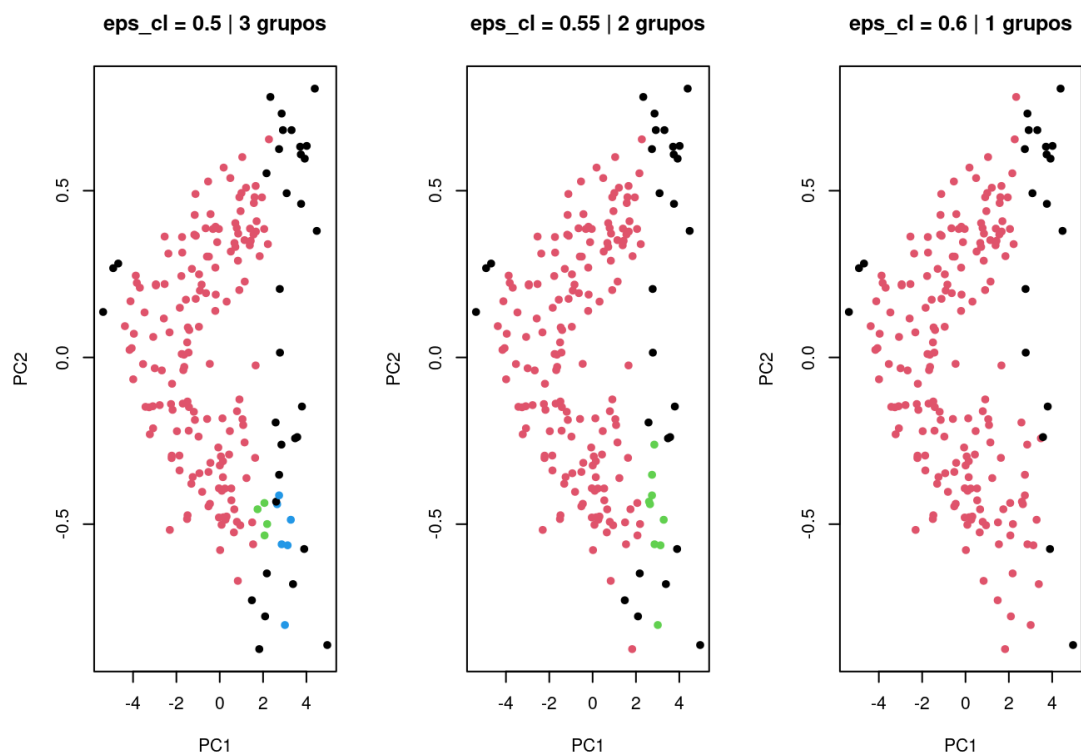
Reachability Plot





```
eps_cortes <- c(0.5, 0.55, 0.6)

par(mfrow = c(1, length(eps_cortes)))
for (e in eps_cortes) {
  oc <- extractDBSCAN(opt_crabs, eps_cl = e)
  col_oc <- ifelse(oc$cluster == 0, "black", oc$cluster + 1)
  plot(X_crabs_pca,
       col = col_oc,
       pch = 19,
       cex = 0.8,
       main = paste0("eps_cl = ", e, " | ", max(oc$cluster), " grupos"),
       xlab = "PC1",
       ylab = "PC2")
}
```



```
par(mfrow = c(1, 1))
```

Cierre

La comparación entre métodos queda así:

k-means:

- simple y útil
- necesita k
- funciona mejor con grupos compactos

GMM:

- modela grupos como gaussianas
- permite elipses e incertidumbre
- usa BIC/ICL para comparar modelos

DBSCAN:

- agrupa por densidad conectada
- no necesita k
- detecta ruido
- depende de eps y minPts

OPTICS:

- explora estructura de densidades
- útil cuando hay densidades distintas
- usa reachability plot y cortes posteriores

La idea importante no es memorizar comandos, sino reconocer qué entiende cada método por "grupo".